

Epipolar Geometry

- Point X projects to p and p'
- The plane formed by X , o , and o' is the epipolar plane.
- Epipoles e , e' are where the baseline intersects the image planes.
- Epipole is the projection of other camera origin in the image plane; focus of expansion / optical center of the camera.
- Epipolar Lines go between the epipoles and projections of the points; out from the optical center.
- Epipoles are finite, maybe in image; epipolar lines converge. Epipolar lines come in pairs.

Essential Matrix

- $E^T e' = 0$ and $E e = 0$
 - epipoles are the nullspace of E - note all epipolar lines pass through epipoles
- Degrees of freedom (Recall $E = [t_x]R$)?
 - 5: 3 (from R) + 3 (from t) - 1 due to scale ambiguity
- E is singular (rank 2); it has two non-zero and identical singular values

Fundamental Matrix

- $Fp, F^T p'$ are epipolar lines for p, p'
- $F e = 0, F^T e' = 0$
- F is singular (rank 2)
- F has seven degrees of freedom
- F definitely not unique

Eight Point Algorithm

Each point gives an equation:

$$\begin{bmatrix} u_1 u_1' & u_1 v_1' & u_1 & u_1 v_1' & v_1 v_1' & v_1 & 1 \end{bmatrix} \cdot \begin{bmatrix} f_{11} & f_{12} & f_{13} & f_{21} & f_{22} & f_{23} & f_{31} & f_{32} & f_{33} \end{bmatrix} = 0$$

Stack equations to yield U:

$$U = \begin{bmatrix} u_1 u_1' & u_1 v_1' & u_1 & u_1 v_1' & v_1 v_1' & v_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_n u_n' & u_n v_n' & u_n & u_n v_n' & v_n v_n' & v_n & 1 \end{bmatrix}$$

Given correspondences $p = [u, v, 1]$ and $p' = [u', v', 1]$ (e.g., via SIFT) we know: $p^T F p' = 0$

$$\begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix} = 0 \Rightarrow \begin{bmatrix} u u' & u v' & u & u v' & v v' & v & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0$$

How do we solve for F?

Normalized 8-Point

- Normalize Coordinates
 - Center the image data at the origin, and scale it so the mean squared distance between the origin and the data points is 2 pixels
- Apply 8-point algorithm
 - Use the eight-point algorithm to compute F from the normalized points
- Enforce the rank-2 constraint
 - For example, take SVD of F and throw out the smallest singular value
- Unnormalize Coordinates
 - Transform fundamental matrix back to original units: if T and T' are the normalizing transformations in the two images, then the fundamental matrix in original coordinates is $T'^T F T$

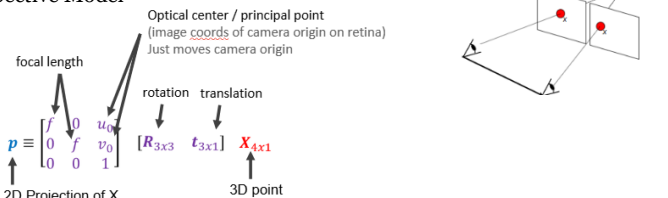
How to solve for F (F unraveled)?:

$$\arg \min_{\|F\|=1} \|U F\|_F^2 \rightarrow \text{Eigenvector of } U^T U \text{ with smallest eigenvalue}$$

Stereo

- Stereo: given two calibrated cameras in different views and correspondences, can solve for X
- Cues: shading, texture, perspective effects, motion, familiar objects
- Calibration: figure out intrinsics of camera (to determine light rays)
- Motion: Figure out R, t for a set of cameras given correspondences
- Recovering Structure: Figure out 3D structure given cameras and correspondences
- Stereo/Epipolar Geometry: Figure out world coordinate for known point given 2 cameras and correspondence

Perspective Model



Camera Calibration

$$\begin{bmatrix} 0^T & -x_1^T & v_1 x_1^T \\ x_1^T & 0^T & -u_1 x_1^T \\ -v_1 x_1^T & u_1 x_1^T & 0^T \end{bmatrix} \begin{bmatrix} m_1^T \\ m_2^T \\ m_3^T \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

We want: $p \equiv K_{3 \times 3} [R_{3 \times 3}, t_{3 \times 1}] X_{4 \times 1}$
 We get: $p \equiv M_{3 \times 4} X_{4 \times 1}$

How to compute $K[R, t]$ from M?

How many linearly independent equations?

How many equations per $[u, v] + [X, Y, Z]$ pair?

If M is 3×4 , how many degrees of freedom?

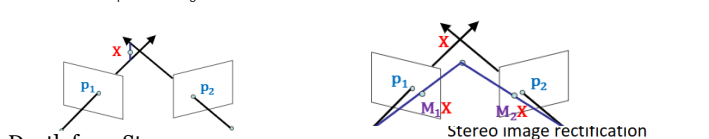
Solution: QR-decomposition on left-most 3×3 matrix $\rightarrow$ finite options of an upper triangular matrix * rotation

Don't forget, ambiguous up to scale!

Find X minimizing $d(p_1, M_1 X)^2 + d(p_2, M_2 X)^2$

Triangulation (2 Options)

Find shortest segment between viewing rays, set X to be the midpoint of the segment.



Depth from Stereo

Essential matrix for parallel images

$$p^T E p = 0 \quad E = [t_x]R$$

What's R?

What's t?

$$R = I \quad t = [T, 0, 0]^T$$

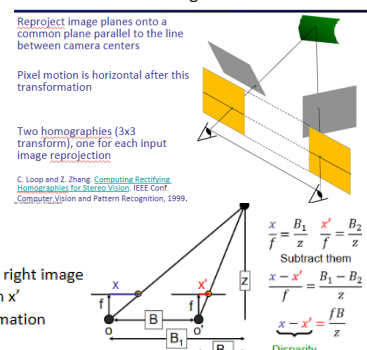
$$E = [t_x]R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -T \\ 0 & T & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -T \\ 0 & T & 0 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = 0 \Rightarrow \begin{bmatrix} 0 \\ 0 \\ -T v + T v \end{bmatrix} = 0 \Rightarrow -T v + T v = 0$$

The y-coordinates of corresponding points are the same!

For each pixel x in the first image

- Find corresponding epipolar scanline in the right image
- Search the scanline and pick the best match x'
- Triangulate matches to get the depth information
 - Depth is inverse to disparity



Machine Learning

Input: x

Supervised: given y

Output: y

Unsupervised: not given y - make our own

Classification / Categorization: bin into k catgeg (discrete / sup)

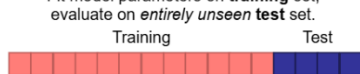
Regression: estimate continuous vars (continuous / sup)

Clustering: given sample, discover clusters/catgeg (discrete / unsup)

Dim Reduction: find dims to explain input (continuous / unsup)

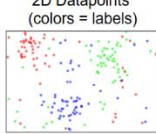
Training and Testing:

Fit model parameters on training set; evaluate on entirely unseen test set.

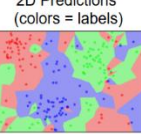


Nearest Neighbor

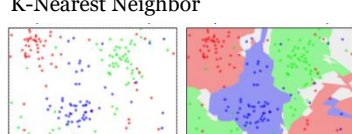
2D Datapoints (colors = labels)



2D Predictions (colors = labels)



K-Nearest Neighbor



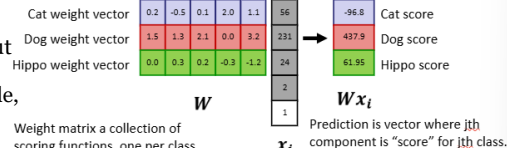
Linear Models

- If w_i is big, then big values of x_i indicate class

Optimization

Grid Search: simple, but scales horribly s^n

Random Search: simple, but slow and may miss



Gradient Descent: at each step, move in the direction of negative gradient. Compute the derivative.

Subgradient: any underestimate of a function (used for abs value, etc)

Step Size / Learning Rate: too small (falls short), too big (diverges).

Momentum: exponentially decaying weights so LR gets smaller (solves issue of oscillation).

Multiple Minima: Gradient Decent finds local minimum. For convex functions, the local minimum is the global minimum.

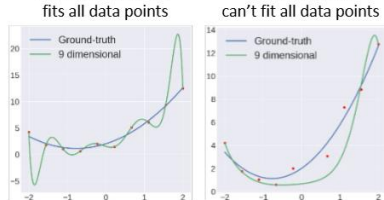
Fitting

Underfitting: model doesn't have parameters to fit the data

Overfitting: high variance; remove one point, model changes majorly

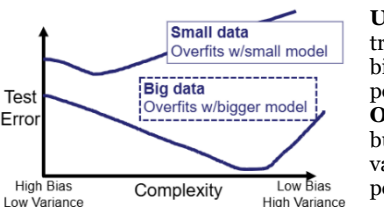
Big weights = more complex model

Regularization:



$$\arg \min_w \|y - Wx\| + \lambda \|w\|$$

Bias: model is oversimplified.
Variance: limited data
Inherent: difficult data
 Bias v. Variance trade-off



Underfitting: poor on training and validation due to bias. More features. More powerful model. Less reg.

Overfitting: well on training but poor on validation due to variance. More data. Less powerful model. Reg more.

Neural Networks

Activation Function: Output of neuron based on inputs

Two-Layer Neural Network: $s = W_2 \max(0, W_1 x)$

Element (i, j) of W_1 gives the effect on h_i from x_j

Element (i, j) of W_2 gives the effect on s from h_j

All elements of x affect all elements of h

All elements of h affect all elements of s

Depth = number of layers

Width: Size of each layer

$$\text{Sigmoid} \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\text{tanh} \quad \tanh(x)$$

$$\text{ReLU} \quad \max(0, x)$$

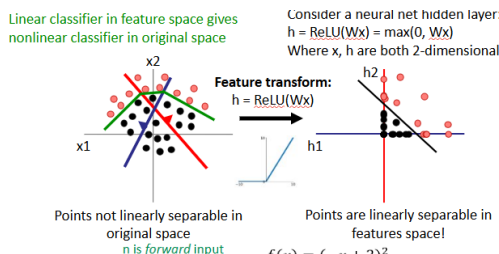
$$\text{Leaky ReLU} \quad \max(0.1x, x)$$

$$\text{Maxout} \quad \max(w_1^T x + b_1, w_2^T x + b_2)$$

$$\text{ELU} \quad \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

$$s = W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

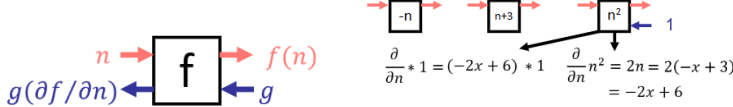
Space Warping



Backprop

Forward: Given forward input n , compute $f(n)$

Backwards: Given backwards input g , return $g * df/dn$



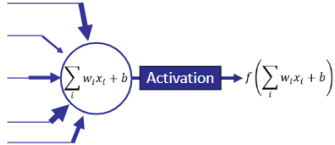
Given two inputs, have two inputs/output wires:

Forward: the same

Backward: the same – send gradients with respect to each variable

Artificial Neuron:

Weighted average of other neuron outputs passed through an activation function



Connect neurons to make a more complex function; use backprop to compute gradient

Activation

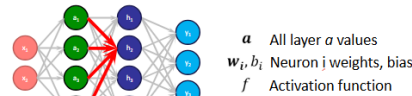
Function

Sigmoid: nice interpretation; squashes to (0,1); gradient near 0 if lo/hi

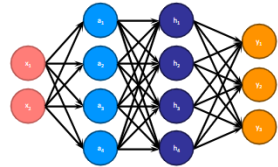
ReLU: constant gradient; conv 6x faster; if neg, then 0 gradient

Leaky ReLU: ReLU but allows small gradients for neg values

Fully Connected Network: each neuron connects to each neuron in previous layer

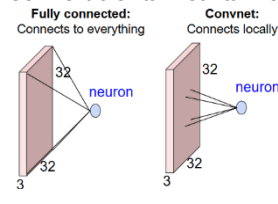


Parameters:

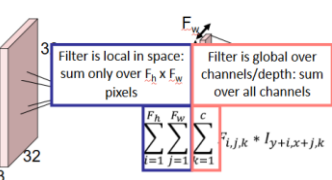


Weights: $3 \times 4 + 4 \times 4 + 4 \times 2 = 36$
Parameters: $36 + 11 = 47$

Convolutional Neural Network (Convnet)



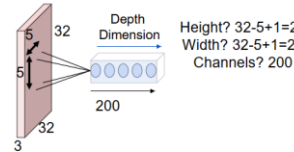
Neuron is the same: weighted linear average



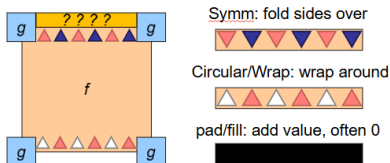
Get spatial output by sliding filter over image.

Multiple output channels via multiple filters.

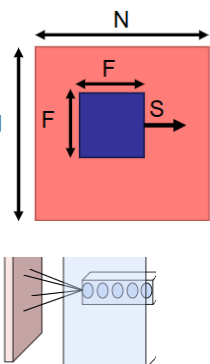
How big is the output?



Zero padding is extremely common, although other forms of padding do happen

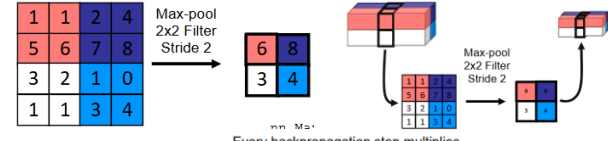


Input volume: $32 \times 32 \times 3$
 Receptive fields: 5×5 , stride 3
 Number of neurons: 5



Output volume: $(32 - 5) / 3 + 1 = 10$, so: $10 \times 10 \times 5$
 How many parameters? $5 \times 5 \times 3 \times 5 + 5 = 380$. Same!

Pooling: spatial resolution of activations / image smaller; applied per-channel

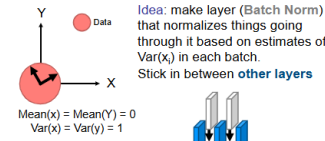


Classic Pipeline of hand-engineered steps

Deep Pipeline of learned convolutions + simple operations

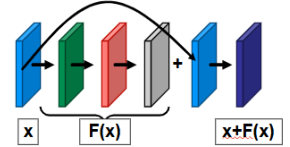
What are some differences?

The classic steps don't talk to each other or have many parameters that are learned from data.



$d \gg 1 \rightarrow$ exploding gradient
 New Building Block: $x + F(x)$

Lets you train networks with 100s of layers



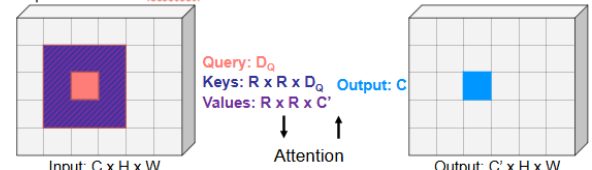
Data Augmentation: apply transformations that do not effect the output

Fine Tuning: (1) extract some layer from existing network, (2) use as new feature, and (3) learn a linear model. OR Transfer learning (steal) <10k img: features; always try finetuning; >100k img (scratch)

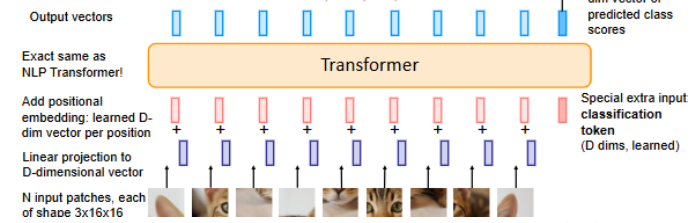
Self-Attention: enables referencing across long sequences

Map center of receptive field to **query**
 Map each element in receptive field to **key** and **value**
 Compute **output** using attention
 Replace all conv in ResNet with local attention

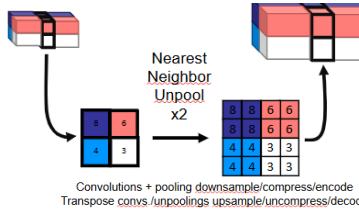
Lots of tricky details, hard to implement, only marginally better than ResNets



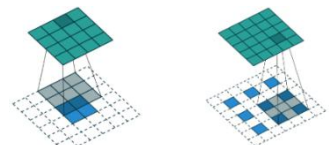
Computer vision model with no convolutions! (not quite)



Unpooling



Transpose Convolution



Encode to spatially smaller tensor, then decode. Encode to 1D vector, then decode.

R-CNN: accurate, modular; slow, ad-hoc training objectives
Fast R-CNN: region proposals + RoI polling on top of a conv feature

Faster R-CNN: RPN + RoI polling